

搭配 TensorFlow.js，在 Node 使用 Coral Edge TPU 執行 TFlite 模型

來源：<https://codelabs.developers.google.com/tensorflowjs-coral-tflite-node?hl=zh-tw>

步驟數：7

在 Node.js 中執行 TensorFlow Lite 模型，並使用 Coral Edge TPU 和 WebNN 加快模型速度。

目錄

1. [簡介](#)
2. [做好準備](#)
3. [訓練圖片分類器](#)
4. [在應用程式中執行 CPU 模型](#)
5. [在應用程式中執行 Coral 模型](#)
6. [使用 WebNN 加速 CPU 模型](#)
7. [恭喜](#)

1. 簡介



上次更新時間：2022 年 4 月 11 日

在本程式碼實驗室中，您將瞭解如何使用 訓練學習機器 訓練圖片分類模型，並透過 [TensorFlow.js](#) (適用於 JavaScript 的強大彈性機器學習程式庫)，使用 [Coral 硬體加速](#) 功能運作執行模型。您將建構 Electron 應用程式，顯示網路攝影機的圖片，並使用 Coral Edge TPU 分類。您可以在 [sig-tfjs GitHub 存放區](#) 中找到本程式碼研究室的 [完整運作版本](#)。

我需要 Coral 裝置嗎？

不需要。您可以在沒有 Coral 裝置的情況下試用這個程式碼研究室，並改用 WebNN 加速器，在桌上型電腦上獲得良好效能。

建構項目

在本程式碼研究室中，您將建構可分類圖片的 Electron 應用程式。您的應用程式：

- 將網路攝影機的圖片分類到您訓練模型時定義的類別。
- 如果有的話，使用 Coral 加速器提升效能。
- 如果平台支援 WebNN，就會使用這項技術提升效能。

課程內容

- 如何安裝及設定 [tfjs-tflite-node](#) NPM 套件，在 Node.js 中執行 TFLite 模型。
- 如何安裝 Edge TPU 執行階段程式庫，在 Coral 裝置上執行模型。
- 如何使用 Coral Edge TPU 加速模型推論。
- 如何使用 [WebNN](#) 加速模型推論。

本程式碼研究室著重於 Node.js 中的 TFLite。我們不會對與主題無關的概念和程式碼多做介紹，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上即可。

軟硬體需求

如果要完成本程式碼研究室，請務必符合以下條件：

- 配備網路攝影機的電腦。
- 如果是 Coral，建議使用搭載 [Raspberry Pi OS \(64 位元\)](#) 的 [Raspberry Pi](#)，並搭配桌面。
- 如果是 WebNN，建議使用執行 Ubuntu 20.04 或 Windows 10 的 Intel x86-64 電腦。
- [Node.js](#) 版本 ≥ 12 。

- 具備 JavaScript 知識。
 - (建議) [Coral USB Accelerator](#)，可加快模型運作速度。
-

步驟 2 · 約 3 分鐘

2. 做好準備

取得程式碼

我們已將本專案所需的程式碼都放到 Git 存放區中。如要開始進行本專案，請先取得程式碼並在慣用的開發環境中開啟。在本程式碼研究室中，我們建議使用搭載 [Raspberry Pi OS \(64 位元\)](#) 的 [Raspberry Pi](#)，並使用桌面環境。方便您連結 Coral 加速器。

強烈建議：使用 Git 在 Raspberry Pi 上複製存放區

如要取得程式碼，請開啟新的終端機視窗並複製存放區：

〇〇〇〇

```
git clone https://github.com/tensorflow/sig-tfjs.git
```

您需要在程式碼研究室中編輯的所有檔案，都位於 `tfjs-tflite-node-codelab` 目錄 (位於 `sig-tfjs` 內)。在這個目錄中，您會看到名為 `starter_code`、`cpu_inference_working`、`coral_inference_working` 和 `webnn_inference_working` 的子目錄。這些是本程式碼研究室各步驟的檢查點。

存放區中的其他檔案包括 `tfjs-tflite-node-codelab` 依附的 NPM 套件。您不需要編輯任何這些檔案，但必須執行部分測試，確保環境設定正確。

安裝 Edge TPU 執行階段程式庫

使用 Coral 裝置前，請先安裝 Edge TPU 執行階段程式庫。請按照所用平台適用的指示安裝。

Linux / Raspberry Pi

在 Linux 上，這個程式庫可透過 Google 的 PPA 取得，是適用於 x86-64 和 Armv8 (64 位元) 架構的 [Debian 套件 libedgetpu1-std](#)。如果處理器使用不同的架構，您需要[從來源編譯](#)。

執行這項指令，新增 Google 的 Coral PPA 並安裝 Edge TPU 執行階段程式庫。

```
# None of this is needed on Coral boards
# This repo is needed for almost all packages below
echo "deb https://packages.cloud.google.com/apt coral-edgetpu-stable main" | sudo tee
/etc/apt/sources.list.d/coral-edgetpu.list
# This repo is needed for only python3-coral-clouddiot and python3-coral-enviro
echo "deb https://packages.cloud.google.com/apt coral-cloud-stable main" | sudo tee
/etc/apt/sources.list.d/coral-cloud.list

curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -

sudo apt-get update
sudo apt-get install libedgetpu1-std
```

在 Windows / 其他作業系統上

預先編譯的二進位檔適用於 [MacOS](#) 和 [Windows](#) 的 [x86-64 版本](#)，下載後執行封存檔案中的 `install.sh` 或 `install.bat` 指令碼即可安裝。

重新啟動裝置

安裝 Edge TPU 執行階段後，請重新啟動裝置，啟用安裝程式新增的 Coral Udev 規則。

確認系統已偵測到 Coral 裝置

如要確認系統是否偵測到 Coral 裝置並正常運作，請執行 `coral-tflite-delegate` 套件的整合測試。這個套件位於存放區的根目錄中。如要執行整合測試，請插入 Coral 加速器，並在套件的目錄中執行下列指令：

```
npx yarn
npx yarn build-deps
npx yarn test-integration
```

注意：如果沒有 `npx`，請確認已安裝 Node.js。您可以按照 nodejs.org 上的操作說明安裝 Node。請務必安裝 12 以上版本。執行 `apt install nodejs` 可能會安裝 Node 10，但本程式碼研究室不應使用這個版本。

畫面會顯示類似如下的輸出：

```
yarn run v1.22.17
$ yarn build && yarn test-integration-dev
$ tsc
$ jasmine --config=jasmine-integration.json
Platform node has already been set. Overwriting the platform with node.
Randomized with seed 78904
Started

=====
Hi there 🙌. Looks like you are running TensorFlow.js in Node.js. To speed things up
dramatically, install our node backend, which binds to TensorFlow C++, by running npm i
@tensorflow/tfjs-node, or npm i @tensorflow/tfjs-node-gpu if you have CUDA. Then call
require('@tensorflow/tfjs-node'); (-gpu suffix for CUDA) at the start of your program. Visit
https://github.com/tensorflow/tfjs-node for more details.
=====
WARNING: converting 'int32' to 'uint8'
.

1 spec, 0 failures
Finished in 2.777 seconds
Randomized with seed 78904 (jasmine --random=true --seed=78904)
Done in 6.36s.
```

請放心，您不必按照記錄檔中的說明安裝 `@tensorflow/tfjs-node`，因為您將在 TFLite 中執行模型。

如果輸出內容包含 `Encountered unresolved custom op: edgetpu-custom-op`，表示系統未偵測到 Coral 裝置。請確認你已安裝 Edge TPU 執行階段程式庫，並將 Coral 裝置插入電腦。您也可以按照 [Coral 的入門指南](#)，測試 Python 版本的 Coral 繫結。如果 Python 版本正常運作，但這些測試仍失敗，請[提出錯誤報告](#)，讓我們瞭解情況。

執行範例程式碼

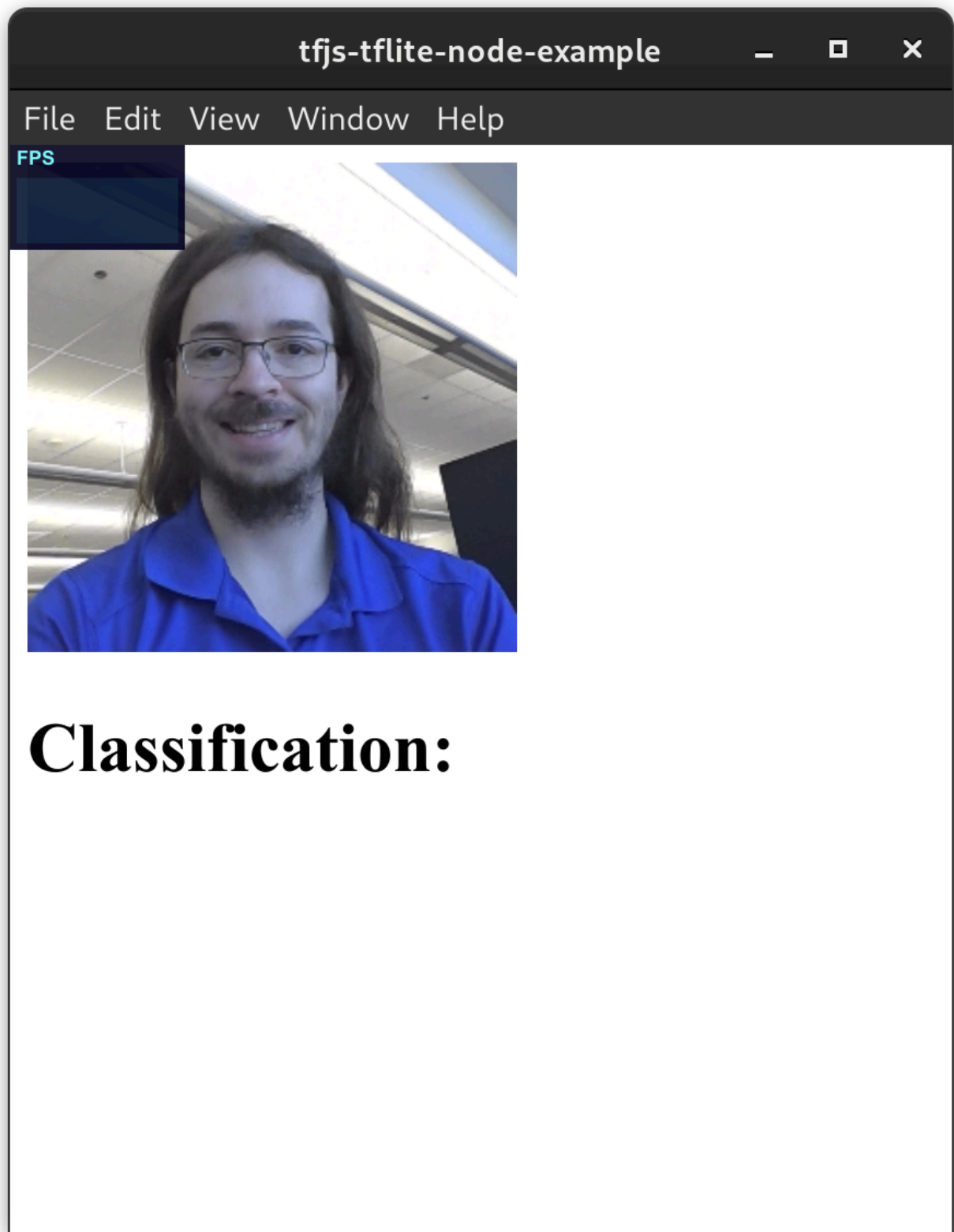
現在可以執行範例程式碼了。如要開始使用，請按照下列步驟操作。

1. 移至 `tfjs-tflite-node-codelab` 目錄下的 `starter_code` 目錄。
2. 執行 `npm install` 安裝依附元件。
3. 執行 `npm start` 啟動專案。應用程式應會開啟，並顯示電腦網路攝影機的影像畫面。

注意：如果您使用 Coral 開發板，且 `npm install` 擲回 `ENOMEM` 錯誤，請執行 `npm rebuild && npm install`，重建損毀的套件。

從哪裡開始？

我們要從專為本次程式碼研究室所設計的基本 [Electron](#) 相機應用程式開始。程式碼已簡化，僅用於說明程式碼研究室中的概念，且幾乎沒有錯誤處理機制。如果您選擇在正式版應用程式中重複使用任何程式碼，請務必處理所有錯誤，並完整測試所有程式碼。



探索範例程式碼

範例程式碼中有很多檔案，但您只需要編輯 `renderer.js`。這個檔案會控管網頁上顯示的內容，包括影片動態消息和 HTML 元素，您可以在這裡將機器學習模型新增至應用程式。其他檔案中包含 `index.html` 檔案，但這個檔案只會載入 `renderer.js` 檔案。此外，還有 `main.js` 檔案，這是 [Electron](#) 的進入點。這個檔案會控管應用程式的生命週期，包括開啟時要顯示的內容，以及關閉時要執行的動作，但您不需要對這個檔案進行任何變更。

開啟偵錯工具

按照本程式碼研究室的指示操作時，您可能需要對應用程式進行偵錯。由於這個應用程式是以 [Electron](#) 為基礎，因此內建 Chrome 除錯工具。在大多數平台上，您可以使用 **Ctrl + Shift + i** 開啟。按一下「控制台」分頁標籤，即可查看應用程式的記錄和錯誤訊息。

這裡沒什麼好探索的，我們直接開始訓練圖像分類器吧！

3. 訓練圖片分類器

在本節中，您將訓練自訂圖片分類模型的 TFLite 和 Coral 版本。

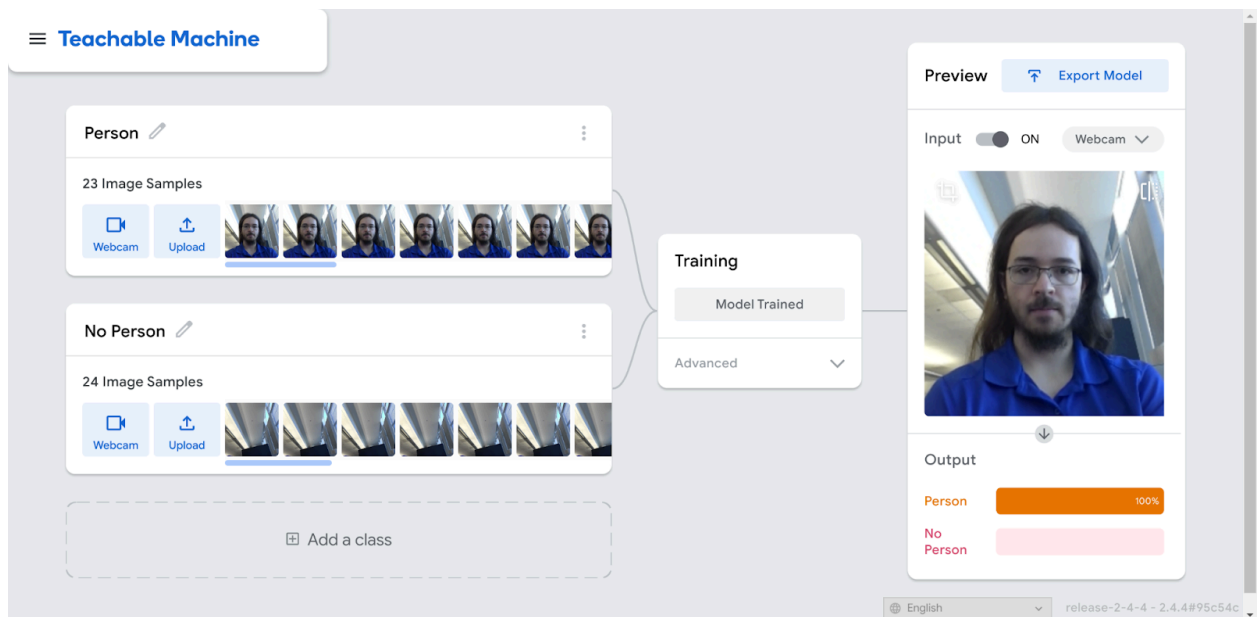
訓練分類器

圖像分類器會接收輸入圖片，並為圖片指派標籤。在本程式碼研究室中，您會使用 [訓練學習機器](#) 在瀏覽器中訓練模型。如要加快本節的訓練速度，可以使用桌上型電腦或筆電，不必使用 Raspberry Pi，但必須將產生的檔案複製到 Pi。

現在可以訓練模型了。如果不確定要訓練哪種模型，可以先訓練人員偵測器，這類模型只會偵測畫面中是否有人。

1. 在新分頁中開啟 [Teachable Machine 訓練頁面](#)。
2. 選取「Image Project」（圖片專案），然後選取「Standard image model」（標準圖片模型）。
3. [為每個類別新增圖片樣本](#)。最簡單的方法是使用網路攝影機輸入。您也可以重新命名課程。
4. 為每個類別收集足夠的資料（通常 50 個樣本就夠了）後，請按下「訓練模型」。

模型訓練完成後，您應該會看到模型輸出的預覽畫面。



請嘗試提供不同的輸入內容。如果發現輸入內容分類錯誤，請將其加入訓練資料並重新訓練模型。

5. 如果對模型的準確度感到滿意，請按一下「匯出模型」。你必須下載兩個不同版本的模型。
6. 將模型匯出為 Tensorflow Lite 浮點模型。系統會下載名為 `converted_tflite.zip` 的檔案。在 CPU 上執行的作業。
7. 將模型匯出為 Tensorflow Lite EdgeTPU 模型。這會下載名為 `converted_edgetpu.zip` 的檔案，可在 Coral Edge TPU 上執行。

提示：為提高準確度，請在圖片樣本中使用各種旋轉角度、位置和背景。如有筆電，建議使用筆電收集這些樣本，會比較輕鬆。

步驟 4 · 約 0 分鐘

4. 在應用程式中執行 CPU 模型

現在您已訓練模型，接著要將模型新增至應用程式。完成本節後，應用程式就能使用裝置的 CPU 執行模型。

將模型檔案新增至應用程式

將訓練分類器時下載的 `converted_tflite.zip` 模型檔案解壓縮。封存檔中有兩個檔案。`model_uquant.tflite` 是儲存的 TFLite 模型，包括模型圖和權重。`labels.txt` 包含模型預測的類別，以及清楚易懂的標籤。將這兩個檔案放在 [modeldirectory](#) 中。

安裝依附元件

載入模型和預先處理輸入內容需要 TensorFlow.js 的幾個依附元件：

- `tfjs-tflite-node`：TensorFlow.js 的套件，可在 Node.js 中執行 TFLite 模型。
- `@tensorflow/tfjs`：TensorFlow.js 的主要套件。

`@tensorflow/tfjs` 已安裝完成，但您需要使用下列指令安裝 `tfjs-tflite-node`：

```
npm install --save tfjs-tflite-node
```

安裝完成後，請在 `renderer.js` 頂端將其新增至應用程式：

[程式碼研究室第 1 部分](#)：匯入 `tfjs-tflite-node`。

```
const {loadTFLiteModel} = require('tfjs-tflite-node');
```

載入模型

現在可以載入模型了。`tfjs-tflite-node` 提供 `loadTFLiteModel` 函式來執行這項操作。它可以從檔案路徑、ArrayBuffer 或 TFHub 網址載入模型。如要載入模型及其權重，請在 `main` 函式中加入下列程式碼：

[程式碼研究室第 1 部分](#)：在此載入模型。

```
const modelPath = './model/model_unquant.tflite';
const model = await loadTFLiteModel(modelPath);
const labels = fs.readFileSync('./model/labels.txt', 'utf8')
    .split('\n');
```

執行模型

執行模型需要三個步驟。首先，從網路攝影機擷取並預先處理輸入影格。接著，您可以在該影格上執行模型並取得預測結果。接著，在網頁上顯示預測結果。

預先處理網路攝影機輸入內容

目前網路攝影機只是 HTML 元素，JavaScript 轉譯器.js 檔案無法使用顯示的影格。如要從網路攝影機擷取影格，TensorFlow.js 提供 [tf.data.webcam](#)，其中包含易於使用的 `capture()` 方法，可從攝影機擷取影格。

如要使用這項功能，請將設定碼新增至 `main()`：

[程式碼研究室第 1 部分：在此設定 `tf.data.webcam`。](#)

```
const tensorCam = await tf.data.webcam(webcam);
```

接著，如要擷取每個影格的圖片，請在 `run()` 中新增下列內容：

[程式碼研究室第 1 部分：在這裡擷取網路攝影機畫面。](#)

```
const image = await tensorCam.capture();
```

此外，您也需要預先處理每個影格，確保影格與模型相容。本程式碼研究室使用的模型輸入形狀為 `[1, 224, 224, 3]`，因此需要 `224 x 224` 像素的 RGB 圖片。`tensorCam.capture()` 的形狀為 `[224, 224, 3]`，因此您需要在張量前端使用 `tf.expandDims` 新增額外維度。此外，[CPU 模型預期輸入介於 -1 和 1 之間的 `Float32`](#)，但網路攝影機擷取的值介於 0 到 255 之間。您可以將輸入張量除以 127，將範圍從 `[0, 255]` 變更為 `[0, ~2]`，然後減去 1，即可取得所需的範圍 `[-1, ~1]`。如要這麼做，請在 `run()` 函式的 `tf.tidy()` 中新增下列幾行程式碼：

[程式碼研究室第 1 部分：在此預先處理網路攝影機影格。](#)

```
const expanded = tf.expandDims(image, 0);
const divided = tf.div(expanded, tf.scalar(127));
const normalized = tf.sub(divided, tf.scalar(1));
```

使用張量後，請務必將其處置。`tf.tidy()` 會自動為回呼中包含的程式碼執行這項操作，但不支援非同步函式。您需要呼叫 `dispose()` 方法，手動處置先前建立的圖片張量。

[程式碼研究室第 1 部分：在此處處置網路攝影機影格。](#)

```
image.dispose();
```

執行模型並顯示結果

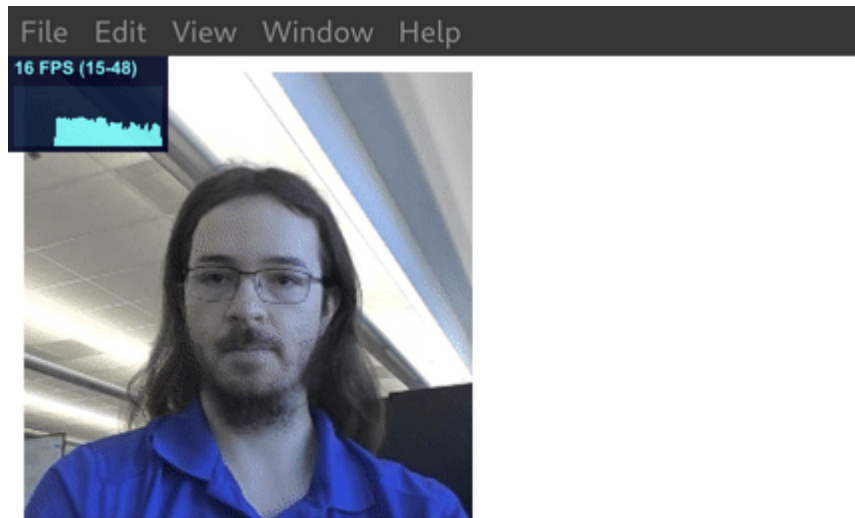
如要在預先處理的輸入內容上執行模型，請在標準化張量上呼叫 `model.predict()`。這會傳回一維張量，其中包含每個標籤的預測機率。將這個機率乘以 100，即可取得每個標籤的百分比機率，並使用範例程式碼隨附的 `showPrediction` 函式，在畫面上顯示模型的預測結果。

這段程式碼也會使用 `stats.js`，在 `model.predict` 周圍呼叫 `stats.begin` 和 `stats.end`，計算預測作業所需時間。

[程式碼研究室第 1 部分：執行模型並在此顯示結果。](#)

```
stats.begin();
const prediction = model.predict(normalized);
stats.end();
const percentage = tf.mul(prediction, tf.scalar(100));
showPrediction(percentage.dataSync(), labels);
```

再次執行應用程式 (使用 `yarn start`)，您應該會看到模型提供的分類。



Classification:

- 0 Person: 100.0
- 1 No Person: 0.0

效能

目前設定的模型是在 CPU 上執行，這對桌上型電腦和大多數筆電來說沒問題，但如果您是在 Raspberry Pi 或其他低功耗裝置上執行，可能就不太理想。在 Raspberry Pi 4 上，您可能會看到大約 10 FPS，這對某些應用程式來說可能不夠快。如要提升效能，但不想使用速度更快的機器，可以採用 Coral Edge TPU 形式的應用程式專用晶片。

步驟 5 · 約 0 分鐘

5. 在應用程式中執行 Coral 模型

如果沒有 Coral 裝置，可以略過這個部分。

本程式碼研究室的這項步驟會以您在上一節中編寫的程式碼為基礎，但如果您想從頭開始，也可以使用 [cpu_inference_working](#) 查核點。

執行 Coral 模型的步驟與執行 CPU 模型的步驟幾乎相同。主要差異在於模型格式。由於 Coral 僅支援 uint8 張量，因此模型會經過量化。這會影響傳遞至模型的輸入張量，以及模型傳回的輸出張量。另一個差異是，模型必須使用 Edge TPU 編譯器編譯，才能在 Coral TPU 上執行。TeachableMachine 已完成這個步驟，但如要瞭解如何為其他模型執行這項操作，請參閱 [Coral 說明文件](#)。

將 Coral 模型檔案新增至應用程式

將訓練分類器時下載的 **converted_edgetpu.zip** 模型檔案解壓縮。封存檔包含兩個檔案。`model_edgetpu.tflite` 是儲存的 TFLite 模型，包括模型圖和權重。`labels.txt` 包含模型預測的類別標籤 (清楚易懂)。將模型檔案放在 **coral_model** 目錄中。

安裝依附元件

執行 Coral 模型需要 [Edge TPU 執行階段程式庫](#)。請先按照設定說明安裝，再繼續操作。

Coral 裝置會以 [TFLite 委派](#) 的形式存取。如要從 JavaScript 存取這些函式，請安裝 [coral-tflite-delegate](#) 套件：

```
npm install --save coral-tflite-delegate
```

接著，在 **renderer.js** 檔案頂端新增這行，匯入委派：

[程式碼研究室第 2 部分](#)：在此匯入委派。

```
const {CoralDelegate} = require('coral-tflite-delegate');
```

載入模型

現在可以載入 Coral 模型了。您會以與 CPU 模型相同的方式執行這項操作，但現在要將選項傳遞至 `loadTFLiteModel` 函式，載入 Coral 委派。

[程式碼研究室第 2 部分](#)：在此載入委派模型。

```
const coralModelPath = './coral_model/model_edgetpu.tflite';
const options = {delegates: [new CoralDelegate()]};
const coralModel = await loadTFLiteModel(coralModelPath, options);
```

標籤與 CPU 模型相同，因此不需要載入。

新增按鈕，在 CPU 和 Coral 之間切換

您要新增 Coral 模型，以及在上一個部分新增的 CPU 模型。同時執行兩者會難以看出效能差異，因此切換鈕會在 Coral 和 CPU 執行作業之間切換。

使用下列程式碼新增按鈕：

[CODELAB 第 2 部分：在此建立委派按鈕。](#)

```
let useCoralDelegate = false;
const toggleCoralButton = document.createElement('button');
function toggleCoral() {
  useCoralDelegate = !useCoralDelegate;
  toggleCoralButton.innerText = useCoralDelegate
    ? 'Using Coral. Press to switch to CPU.'
    : 'Using CPU. Press to switch to Coral.';
}
toggleCoralButton.addEventListener('click', toggleCoral);
toggleCoral();
document.body.appendChild(toggleCoralButton);
```

我們在 `run()` 函式中連結這個條件。如果 `useCoralDelegate` 為 `false`，則應執行 CPU 版本。否則會執行 Coral 版本 (但目前不會有任何動作)。將執行 **CPU** 模型時的程式碼包裝在 `if` 陳述式中。請注意，由於 Coral 模型會使用 `expanded` 張量，因此 `if` 陳述式中會省略該張量。

[程式碼研究室第 2 部分：檢查是否要在此處使用委派。](#)

```
// NOTE: Don't just copy-paste this code into the app.
// You'll need to edit the code from the CPU section.
const expanded = tf.expandDims(image, 0);
if (useCoralDelegate) {
  // CODELAB part 2: Run Coral prediction here.
} else {
  const divided = tf.div(expanded, tf.scalar(127));
  const normalized = tf.sub(divided, tf.scalar(1));
  stats.begin();
  const prediction = model.predict(normalized);
  stats.end();
  const percentage = tf.mul(prediction, tf.scalar(100));
  showPrediction(percentage.dataSync(), labels);
}
```

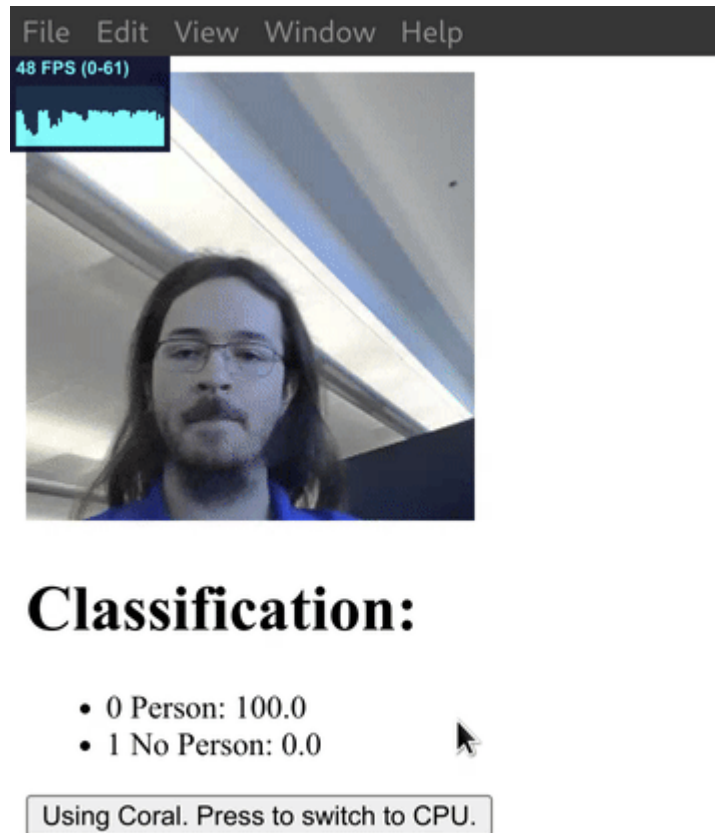
執行模型

模型的 Coral 版本預期會收到介於 0 到 255 的 `uint8` 張量，因此不需要將輸入內容正規化。不過，輸出也是介於 0 到 255 的 `uint8` 張量。顯示前，必須先將其轉換為 0 到 100 的浮點值。

程式碼研究室第 2 部分：在此執行 Coral 預測。(這是上述程式碼片段的一部分)

```
stats.begin();
const prediction = coralModel.predict(expanded);
stats.end();
const percentage = tf.div(tf.mul(prediction, tf.scalar(100)), tf.scalar(255));
showPrediction(percentage.dataSync(), labels);
```

再次執行應用程式 (使用 `yarn start`)，系統應會顯示 Coral 加速器的分類。



按下按鈕即可在 Coral 和 CPU 推論之間切換。你可能會發現，與 CPU 模型相比，Coral 模型的信賴度排名較不精確，而且通常會以偶數的小數位數結尾。在 Coral 上執行量化模型時，精確度會有所損失，實務上通常沒有影響，但請謹記這一點。

效能注意事項

您看到的影格率包含前置處理和後置處理，因此無法代表 Coral 硬體의效能。如要進一步瞭解效能，請按一下 FPS 計量表，直到顯示延遲時間 (以毫秒為單位)，這只會測量對 `model.predict` 的呼叫。不過，這仍包含將 Tensor 移至 TFLite 原生 C 繫結，然後移至 Coral 裝置所需的時間，因此並非完美測量結果。如要查看以 C++ 編寫的更準確效能基準，請參閱 [EdgeTPU 基準頁面](#)。

此外，這部影片是在筆電上錄製，而非 Raspberry Pi，因此你可能會看到不同的 FPS。

加快 Coral 前置處理速度

在某些情況下，切換 TFJS 後端可加快前置處理速度。預設後端是 WebGL，適合用於大型可平行處理的作業，但這個應用程式在預先處理階段中並未執行太多這類作業 (使用的唯一運算作業是 `expandDims`，這項作業無法平行處理)。如要避免將張量移至 GPU 和從 GPU 移出的額外延遲，請在檔案頂端的匯入項目後方新增這行程式碼，切換至 CPU 後端。

```
tf.setBackend('cpu');
```

這也會影響 TFLite CPU 模型的預先處理作業 (已平行化)，因此模型執行速度會因這項變更而大幅變慢。

6. 使用 WebNN 加速 CPU 模型

如果沒有 Coral 加速器，或是想試試其他模型加速方式，可以使用 [WebNN TFLite 委派](#)。這個委派項目會使用 Intel 處理器內建的機器學習硬體，透過 [OpenVINO 工具包](#) 加速模型推論。因此，它有額外的需求，而這些需求並未涵蓋在本程式碼研究室的設定部分，您需要安裝 OpenVINO 工具包。請務必先根據支援的 [目標系統平台](#) 檢查設定，再繼續操作，但請注意，WebNN 委派目前不支援 macOS。

安裝 OpenVINO 工具包

OpenVINO 工具包會使用 Intel 處理器內建的機器學習硬體加速模型。您可以從 Intel [下載預先編譯的版本](#)，或從[來源建構](#)。安裝 OpenVINO 的方法有很多種，但為了本程式碼研究室的目的，我們建議使用 [Windows](#) 或 [Linux](#) 的安裝程式指令碼。請務必安裝 **2021.4.2 LTS** 執行階段版本，因為其他版本可能不相容。執行安裝程式後，請務必按照 [Linux](#) 或 [Windows](#) 的安裝說明 ([永久解決方案](#)) 設定殼層的環境變數，或執行 `webnn-tflite-delegate` 目錄中的 `setupvars.sh` (Linux) 或 `setupvars.bat` (Windows) 指令。

Windows 注意事項：`setupvars.bat` 指令不支援 Git bash，因此請務必從 Windows 命令提示字元執行這項指令，以及本程式碼研究室中的其他指令。

確認 WebNN 委派是否正常運作

如要驗證 WebNN 委派是否正常運作，請執行存放區根目錄中的 `webnn-tflite-delegate` 套件整合測試。如要執行整合測試，請在套件的目錄中執行下列指令：

```
# In webnn-tflite-delegate/  
npx yarn  
npx yarn test-integration
```

畫面會顯示類似如下的輸出：

```
WebNN delegate: WebNN device set to 0.
INFO: Created TensorFlow Lite WebNN delegate for device Default and power Default.

=====
Hi there 🙌. Looks like you are running TensorFlow.js in Node.js. To speed things up
dramatically, install our node backend, which binds to TensorFlow C++, by running npm i
@tensorflow/tfjs-node, or npm i @tensorflow/tfjs-node-gpu if you have CUDA. Then call
require('@tensorflow/tfjs-node'); (-gpu suffix for CUDA) at the start of your program. Visit
https://github.com/tensorflow/tfjs-node for more details.
=====
label: wine bottle
score: 0.934505045413971
.

1 spec, 0 failures
Finished in 0.446 seconds
Randomized with seed 58441 (jasmine --random=true --seed=58441)
Done in 8.07s.
```

如果看到類似下方的輸出內容，表示設定有誤：

```
Platform node has already been set. Overwriting the platform with node.
Randomized with seed 05938
Started
error Command failed with exit code 3221225477.
info Visit https://yarnpkg.com/en/docs/cli/run for documentation about this command.
```

這個輸出內容很可能表示您尚未設定 OpenVINO 的環境變數。目前，您可以執行 `setupvars.sh` (Linux) 或 `setupvars.bat` (Windows) 指令來設定這些變數，但您可能想按照 [Linux](#) 或 [Windows](#) ([永久解決方案](#)) 的操作說明，永久設定這些變數。如果您使用 Windows，請

setupvars.bat

指令不支援 **Git Bash**，因此請務必從 Windows 命令提示字元執行這項指令，以及本程式碼研究室的其他指令。

安裝 WebNN 委派

安裝 OpenVINO 後，您現在可以透過 WebNN 加速 CPU 模型。本程式碼研究室的這一節會以「在應用程式中執行 CPU 模型」一節中編寫的程式碼為基礎。您可以使用這個步驟中編寫的程式碼，但如果已完成 Coral 部分，請改用 [cpu_inference_working](#) 查核點，以便從頭開始。

WebNN 委派的 Node.js 部分會發布在 npmjs 上。如要安裝，請執行下列指令：

```
npm install --save webnn-tflite-delegate
```

接著，在 `renderer.js` 檔案頂端新增這行，匯入委派：

[程式碼研究室第 2 部分](#)：在此匯入委派。

```
const {WebNNDelegate, WebNNDevice} = require('webnn-tflite-delegate');
```

WebNN 委派支援在 CPU 或 GPU 上執行；您可以透過 WebNNDevice 選擇要使用的裝置。

載入模型

現在，您已準備好載入模型，並啟用 WebNN 委派。使用 Coral 時，您必須載入不同的模型檔案，但 WebNN 支援與 TFLite 相同的模型格式。將 WebNNDelegate 新增至傳遞給模型的委派清單，即可啟用：

[程式碼研究室第 2 部分](#)：在此載入委派模型。

```
let webnnModel = await loadTFLiteModel(modelPath, {
  delegates: [new WebNNDelegate({webnnDevice: WebNNDevice.DEFAULT})],
});
```

由於這是同一個模型，因此不需要再次載入標籤。

新增按鈕，在 Tflite CPU 和 WebNN 之間切換

現在模型已準備好使用 WebNN 版本，請新增按鈕，在 WebNN 和 Tflite CPU 推論之間切換。同時執行兩者會難以看出效能差異。

使用這段程式碼新增按鈕 (請注意，這還不會實際切換模型)：

[CODELAB 第 2 部分](#)：在此建立委派按鈕。

```
let useWebNNDelegate = false;
const divElem = document.createElement('div');
const toggleWebNNButton = document.createElement('button');
function toggleWebNN() {
  useWebNNDelegate = !useWebNNDelegate;
  toggleWebNNButton.innerHTML = useWebNNDelegate
    ? 'Using WebNN. Press to switch to TFLite CPU.'
    : 'Using TFLite CPU. Press to switch to WebNN.';
  divElem.hidden = useWebNNDelegate ? false : true;
}

toggleWebNNButton.addEventListener('click', toggleWebNN);
toggleWebNN();
document.body.appendChild(toggleWebNNButton);
document.body.appendChild(divElem);
```

這段程式碼也會新增 `div` 元素，您將在下一節使用這個元素設定 WebNN。

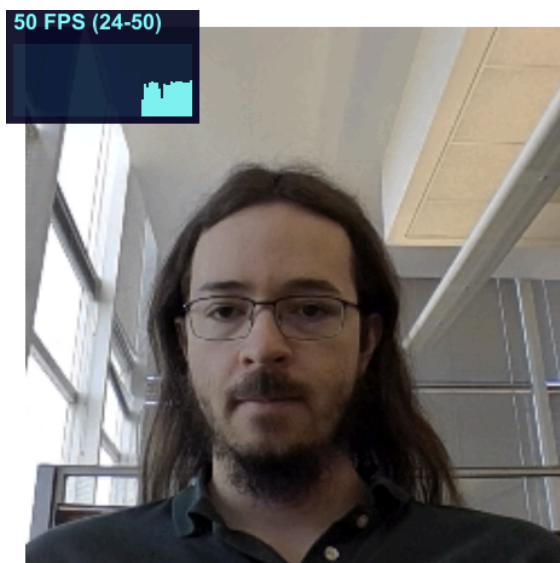
新增下拉式選單，以便在 WebNN 裝置之間切換

WebNN 支援在 CPU 和 GPU 上執行，因此請新增下拉式選單，方便使用者切換。在建立按鈕的程式碼後方新增下列程式碼：

```
// Create elements for WebNN device selection
divElem.innerHTML = '<br/>WebNN Device: ';
const selectElem = document.createElement('select');
divElem.appendChild(selectElem);

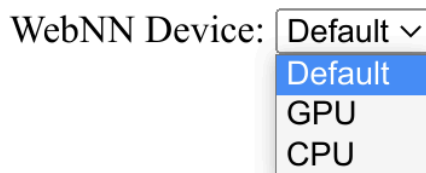
const webnnDevices = ['Default', 'GPU', 'CPU'];
// append the options
for (let i = 0; i < webnnDevices.length; i++) {
  var optionElem = document.createElement('option');
  optionElem.value = i;
  optionElem.text = webnnDevices[i];
  selectElem.appendChild(optionElem);
}
```

現在執行應用程式，您會看到列出「預設」、「GPU」和「CPU」的下拉式選單。由於下拉式選單尚未連結，因此目前選擇任一選項都不會產生任何作用。



Classification:

- 0 Person: 100.0
- 1 No Person: 0.0



讓下拉式選單變更裝置

如要連結下拉式選單，以便變更使用的 WebNN 裝置，請將事件監聽器新增至下拉式選單選取器元素的 **change** 事件。選取的值變更時，請在委派選項中選取對應的 WebNN 裝置，然後重新建立 WebNN 模型。

在新增下拉式選單的程式碼後方，加入下列程式碼：

```
selectElem.addEventListener('change', async () => {
  let webnnDevice;
  switch(selectElem.value) {
    case '1':
      webnnDevice = WebNNDevice.GPU;
      break;
    case '2':
      webnnDevice = WebNNDevice.CPU;
      break;
    default:
      webnnDevice = WebNNDevice.DEFAULT;
      break;
  }
  webnnModel = await loadTFLiteModel(modelPath, {
    delegates: [new WebNNDelegate({webnnDevice})],
  });
});
```

這項變更生效後，每次變更下拉式選單時，系統都會建立具有正確設定的新模型。現在可以連結 WebNN 模型，並用於推論。

執行 WebNN 模型

WebNN 模型已可使用，但切換 WebNN 和 Tflite CPU 的按鈕尚未實際切換模型。如要切換模型，請先重新命名 `model` 變數，這個變數是在程式碼研究室的第一個部分載入 Tflite CPU 模型時建立。

變更下列行...

```
const model = await loadTFLiteModel(modelPath);
```

...與這行相符。

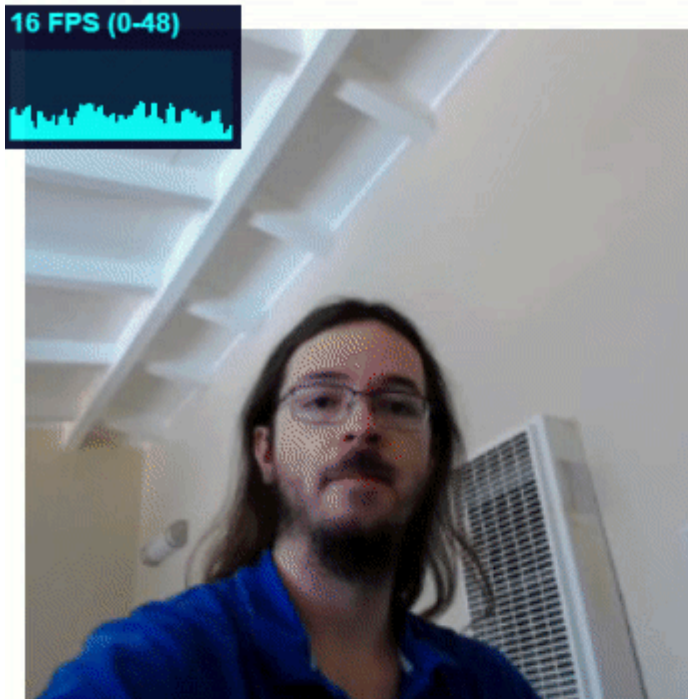
```
const cpuModel = await loadTFLiteModel(modelPath);
```

將 `model` 變數重新命名為 `cpuModel` 後，請將其新增至 `run` 函式，根據按鈕的狀態選擇正確的模型：

[程式碼研究室第 2 部分：檢查是否要在此處使用委派。](#)

```
let model;
if (useWebNNDelegate) {
  model = webnnModel;
} else {
  model = cpuModel;
}
```

現在執行應用程式時，按鈕會在 Tflite CPU 和 WebNN 之間切換。



Classification:

- 0 Person: 99.9
- 1 No Person: 0.1

Using TFLite CPU. Press to switch to WebNN.

如果您有整合式 Intel GPU，也可以在 WebNN CPU 和 GPU 推論之間切換。

效能注意事項

您看到的影格率包含前處理和後處理，因此無法代表 WebNN 的功能。如要進一步瞭解效能，請按一下 FPS 計量表，直到顯示延遲時間 (以毫秒為單位)，這只會測量對 `model.predict` 的呼叫。不過，這仍包含將張量移至 TFLite 原生 C 繫結所需的時間，因此並非完美測量。

步驟 7 · 約 0 分鐘

7. 恭喜

恭喜！您剛才在 Electron 中使用 **tfjs-tflite-node** 完成第一個 Coral / WebNN 專案。

快來試試看，並在各種圖片上測試。您也可以 [在 Teachable Machine 上訓練新模型](#)，分類完全不同的事物。

重點回顧

在本程式碼研究室中，您已瞭解以下內容：

- 如何安裝及設定 **tfjs-tflite-node** npm 套件，在 Node.js 中執行 TFLite 模型。
- 如何安裝 Edge TPU 執行階段程式庫，在 Coral 裝置上執行模型。
- 如何使用 Coral Edge TPU 加速模型推論。
- 如何使用 WebNN 加速模型推論。

後續步驟

您現在已具備可供著手進行的基礎，接下來可以發想哪些創意，將這個機器學習模型執行器擴充到您可能正在處理的實際應用案例？或許您能以快速且經濟實惠的推論功能，徹底改變所屬產業，或是修改烤麵包機，讓它在麵包烤得恰到好處時停止烘烤。一切都有無限的可能。

若要進一步瞭解 Teachable Machine 如何訓練您使用的模型，請參閱[遷移學習](#)程式碼研究室。如要尋找其他適用於 Coral 的模型 (例如語音辨識和姿勢估算)，請前往 [coral.ai/models](#)。您也可以 [在 TensorFlow Hub 找到這些模型的 CPU 版本和許多其他模型](#)。

與我們分享你的作品

您也可以輕鬆將今天製作的內容用於其他創意用途，我們鼓勵您發揮創意，持續探索各種可能性。

別忘了在社群媒體上使用 [#MadeWithTFJS](#) 主題標記，你的專案就有機會登上 [TensorFlow 網誌](#)，甚至在 [日後活動](#) 中展示。我們很期待看到你的作品。

值得一訪的網站

- [TensorFlow.js 官方網站](#)
 - [TensorFlow.js 預先建構模型](#)
 - [TensorFlow.js API](#)
 - [TensorFlow.js 展演秀](#)
-